

Session 1 Lab experiments

Name: Deekshitha U

Date: 05/10/26

1. Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. What is the space complexity of the algorithm?

```
n = int(input("Enter number of elements: "))
```

```
arr = []  
for i in range(n):  
    arr.append(int(input("Enter element: ")))
```

```
unique = []
```

```
for x in arr:  
    if x not in unique:  
        unique.append(x)
```

```
print("Unique elements:", unique)
```

```
1  n = int(input("Enter number of elements: "))
2
3  arr = []
4  for i in range(n):
5      arr.append(int(input("Enter element: ")))
6
7  unique = []
8
9  for x in arr:
10     if x not in unique:
11         unique.append(x)
12
13  print("Unique elements:", unique)
```

Problems Output Debug Console Ports Terminal

```
PS C:\Users\deek\Documents\daa lab> cd 5-10-26
PS C:\Users\deek\Documents\daa lab\5-10-26> python daaexp1.py
Enter number of elements: 5
Enter element: 3
Enter element: 7
Enter element: 9
Enter element: 56
Enter element: 2
Unique elements: [3, 7, 9, 56, 2]
```

2. Sort an array of integers using the bubble sort technique. Analyze its time complexity using Big-O notation. Write the code

```
n = int(input("Enter number of elements: "))
arr = []
for i in range(n):
    arr.append(int(input("Enter element: ")))
# Bubble Sort
for i in range(n - 1):
    for j in range(n - 1 - i):
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]
print("Sorted array:", arr)
```

```
1  n = int(input("Enter number of elements: "))
2
3  arr = []
4  for i in range(n):
5      arr.append(int(input("Enter element: ")))
6
7  # Bubble Sort
8  for i in range(n - 1):
9      for j in range(n - 1 - i):
10         if arr[j] > arr[j + 1]:
11             arr[j], arr[j + 1] = arr[j + 1], arr[j]
12
13  print("Sorted array:", arr)
```

Problems Output Debug Console Ports Terminal

```
PS C:\Users\deek\Documents\daa lab> cd 5-10-26
PS C:\Users\deek\Documents\daa lab\5-10-26> python daaexp2.py
Enter number of elements: 5
Enter element: 8
Enter element: 3
Enter element: 6
Enter element: 7
Enter element: 2
Sorted array: [2, 3, 6, 7, 8]
```

3. Checks if a given number x exists in a sorted array `arr` using binary search. Analyze its time complexity using Big-O notation.

```
n = int(input("Enter number of elements: "))
```

```
arr = []
```

```
for i in range(n):
```

```
    arr.append(int(input("Enter element: ")))
```

```
x = int(input("Enter element to search: "))
```

```
low = 0
```

```
high = n - 1
```

```
found = False
```

```
while low <= high:
```

```
    mid = (low + high) // 2
```

```
    if arr[mid] == x:
```

```
        found = True
```

```
        break
```

```
    elif arr[mid] < x:
```

```
        low = mid + 1
```

```
    else:
```

```
        high = mid - 1
```

```
if found:
```

```
    print("Element found")
```

```
else:
```

```
    print("Element not found")
```

```

1  n = int(input("Enter number of elements: "))
2  arr = []
3  for i in range(n):
4      arr.append(int(input("Enter element: ")))
5
6  x = int(input("Enter element to search: "))
7  low = 0
8  high = n - 1
9  found = False
10 while low <= high:
11     mid = (low + high) // 2
12
13     if arr[mid] == x:
14         found = True
15         break
16     elif arr[mid] < x:
17         low = mid + 1
18     else:
19         high = mid - 1
20 if found:
21     print("Element found")
22 else:
23     print("Element not found")

```

```

PS C:\Users\deek\Documents\daa lab\5-10-26> python daaexp3.py
Enter number of elements: 6
Enter element: 2
Enter element: 4
Enter element: 6
Enter element: 7
Enter element: 9
Enter element: 5
Enter element to search: 4
Element found

```

4. **Given an array of integers nums, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in $O(n\log(n))$ time complexity and with the smallest space complexity possible.**

```
def heapify(nums, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2
    # Check left child
    if left < n and nums[left] > nums[largest]:
        largest = left
    # Check right child
    if right < n and nums[right] > nums[largest]:
        largest = right
    # If largest is not the root
    if largest != i:
        nums[i], nums[largest] = nums[largest], nums[i]
        # Heapify the affected subtree
        heapify(nums, n, largest)

def heap_sort(nums):
    n = len(nums)
    # Build max heap
    for i in range(n // 2 - 1, -1, -1):
        heapify(nums, n, i)
    # Extract elements one by one
    for i in range(n - 1, 0, -1):
        nums[0], nums[i] = nums[i], nums[0]
        heapify(nums, i, 0)
    return nums

# Input
n = int(input("Enter number of elements: "))

nums = []
for i in range(n):
    nums.append(int(input("Enter element: ")))

# Sort
heap_sort(nums)
print("Sorted array:", nums)
```

```

1  def heapify(nums, n, i):
2      largest = i
3      left = 2 * i + 1
4      right = 2 * i + 2
5      # Check left child
6      if left < n and nums[left] > nums[largest]:
7          largest = left
8      # Check right child
9      if right < n and nums[right] > nums[largest]:
10         largest = right
11     # If largest is not the root
12     if largest != i:
13         nums[i], nums[largest] = nums[largest], nums[i]
14
15         # Heapify the affected subtree
16         heapify(nums, n, largest)
17 def heap_sort(nums):
18     n = len(nums)
19     # Build max heap
20     for i in range(n // 2 - 1, -1, -1):
21         heapify(nums, n, i)
22     # Extract elements one by one
23     for i in range(n - 1, 0, -1):
24         nums[0], nums[i] = nums[i], nums[0]
25         heapify(nums, i, 0)
26     return nums
27 # Input
28 n = int(input("Enter number of elements: "))
29 nums = []
30 for i in range(n):
31     nums.append(int(input("Enter element: ")))
32 # Sort
33 heap_sort(nums)
34 print("Sorted array:", nums)

```

```
PS C:\Users\deek\Documents\daa lab\5-10-26> python daaexp4.py
Enter number of elements: 5
Enter element: 6
Enter element: 3
Enter element: 8
Enter element: 2
Enter element: 9
Sorted array: [2, 3, 6, 8, 9]
```